

NAKAST

User Manual

UNIVERSIDAD DE VALPARAÍSO

Authors:

JAVIER ALEJANDRO TOGNARELLI SANTIAGO

DANIEL FERNANDO ESCOBAR ARAYA

FERNANDO ANDRÉS AMAYA INZUNZA

SEPTEMBER 2026

Table of Contents

1. Introduction
2. Access and installation
 - 2.1 Requirements
 - 2.2 Download
 - 2.3 Verifying the installation
 - 2.4 PubMLST access key
 - 2.5 Running without Conda
3. Analysis module: using the pipeline
 - 3.1 A complete worked example
 - 3.2 Input specification
 - 3.3 Parameter reference
 - 3.4 Output reference
 - 3.5 Interpreting your results
 - 3.6 Assessing data quality
4. Technical module: administration and maintenance
 - 4.1 Workflow architecture
 - 4.2 How allele calling works
 - 4.3 ST and CC assignment
 - 4.4 Species support
 - 4.5 Assumptions
 - 4.6 Limitations
 - 4.7 Reproducibility
 - 4.8 Design notes
5. Support, citation and licence
 - 5.1 Reporting problems
 - 5.2 Authorship
 - 5.3 How to cite NAKAST
 - 5.4 References for the components
 - 5.5 Third-party components and data source
6. Frequently asked questions

Una versión en español de este manual está disponible en [MANUAL_USUARIO.md](#).

1. Introduction

NAKAST (*Nanopore Amplicon KMA Allele Sequence Typing*) is a Nextflow pipeline that determines the MLST profile of bacterial isolates from Oxford Nanopore (ONT) amplicon sequencing reads. Given the FASTQ files of each sample and the name of its species, it returns a table with the Sequence Type (ST) of every isolate and, where the scheme defines one, its Clonal Complex (CC).

MLST (*Multi Locus Sequence Typing*) characterises an isolate by the sequence of a small set of housekeeping genes, usually seven. Each distinct sequence at a locus receives an **allele number**; the ordered set of these numbers forms the **allelic profile** (for example 9-1-4-1-3-3-2), and each distinct profile corresponds to a **Sequence Type**. A profile matches an ST only if every allele is an exact match to a known allele. Related STs, which share most of their alleles, are grouped into **clonal complexes**.

Allele and profile definitions come from **PubMLST**, which NAKAST queries on every run to obtain the current scheme for the species. Reads are aligned against those alleles with **KMA**, an aligner designed for redundant databases such as MLST's, able to tell apart alleles that differ by only a few bases.

The pipeline works with any of the 135 PubMLST MLST schemes that define ST profiles. The species is declared per sample, so a single run may include several species: the database is downloaded once per species and one report is produced for each.

Figure 1 summarises the workflow, from validating the sample table to producing the final report.

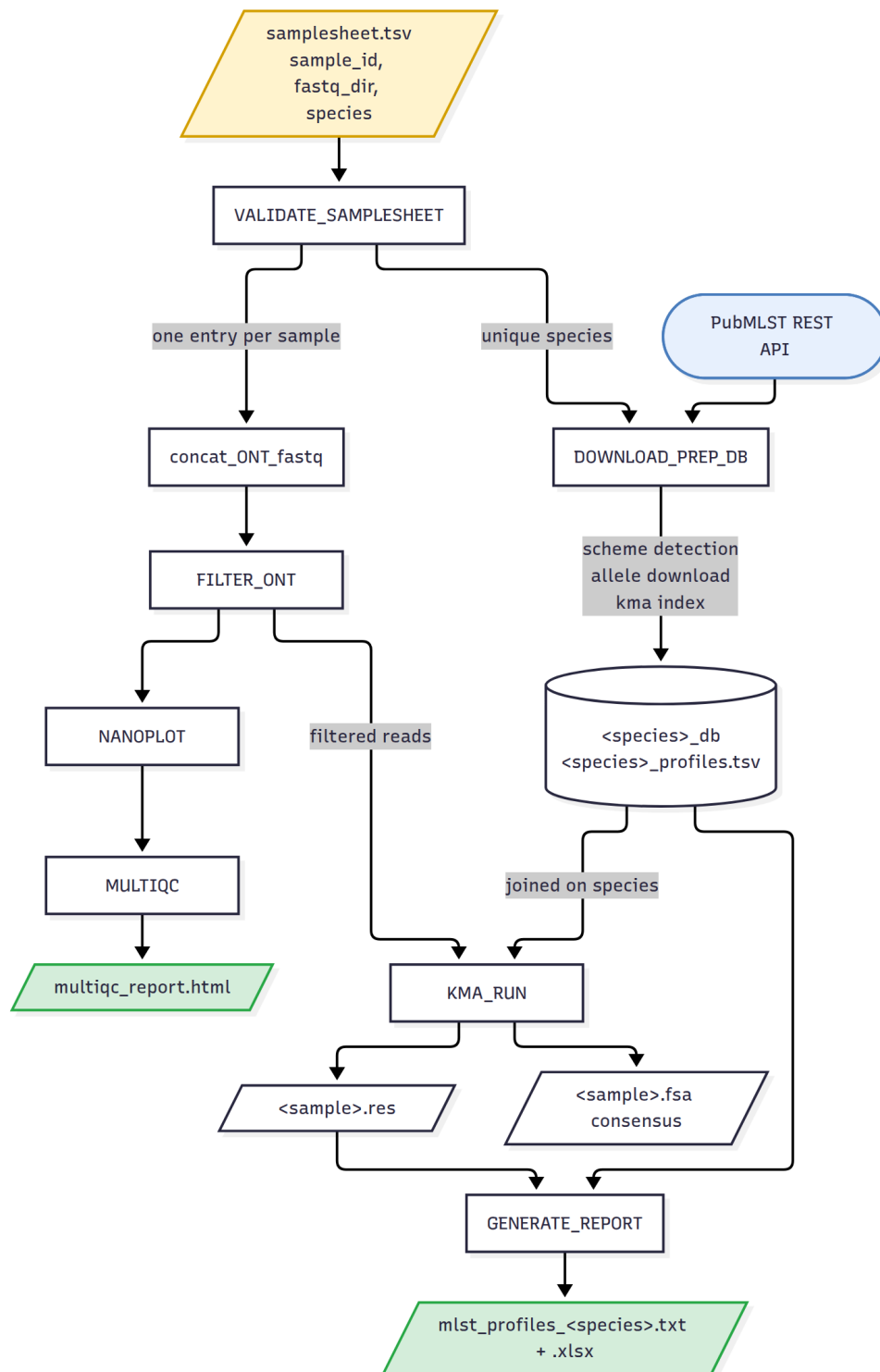


Figure 1. NAKAST workflow. Input in yellow, the external data source in blue and the final results in green.

2. Access and installation

NAKAST requires no registration or user account. It is distributed as source code through its GitHub repository and runs locally from the command line.

2.1 Requirements

Component	Version	Notes
Operating system	Linux	Platform on which it was developed and tested
Nextflow	25.10 or later	Tested on 25.10.2 and 26.04.6
Conda	>= 26.1.1	Miniconda or Miniforge
Mamba	>= 2.4.0	Speeds up building the environments
Internet access	—	PubMLST is queried on every run
PubMLST account	—	Free; its access key gives the complete scheme (section 2.4)

You do not need to install KMA, filtlong, chopper, NanoPlot or MultiQC: each process declares its own Conda environment with pinned versions, and Nextflow builds them on first run.

2.2 Download

```
git clone https://github.com/javiertognarelli/nakast.git
cd nakast
```

2.3 Verifying the installation

```
nextflow run nakast.nf --help
nextflow config .           # should show conda { enabled = true; useMamba = true }
```

The first real run builds the Conda environments, which adds a few minutes. They are stored in `$HOME/.nextflow/conda` and reused on later runs.

2.4 PubMLST access key

Since 2025, PubMLST serves unauthenticated requests only the data deposited up to 31 December 2024. Alleles and STs defined later require an account. NAKAST therefore requires, by default, a PubMLST **data access key**, which it sends with every request.

To obtain one, sign in to your account at <https://pubmlst.org> and create a data access key. Then store it once in the Nextflow secrets store:

```
read -rs -p "PubMLST API key: " K && nextflow secrets set PUBMLST_API_KEY "$K" &&
unset K
```

The command reads the key without echoing it or leaving it in the shell history. Nextflow keeps it in `$HOME/.nextflow/secrets/`, in a file only your user can read, and hands it to the download process without writing it to the work directory, the logs or the results. To confirm it was stored:

```
nextflow secrets list           # should list PUBMLST_API_KEY
```

The key is personal: every user must use their own and never put it in the repository or in configuration files.

Without an account you can still run by adding `--pubmlst_anonymous`. In that mode NAKAST uses only data deposited up to 31 December 2024, and any isolate of an ST or allele defined later will come out untyped.

2.5 Running without Conda

If you prefer to manage the tools yourself, the host profile disables Conda and uses whatever is on your PATH:

```
nextflow run nakast.nf --profile host --samplesheet samplesheet.tsv
```

You then need kma, filtlong, chopper, NanoPlot, multiqc, and Python with pandas, numpy and openpyxl. This is not the recommended mode: it gives up the reproducibility that the pinned environments provide.

3. Analysis module: using the pipeline

This section is for whoever runs the pipeline and interprets its results.

3.1 A complete worked example

A run of three *Streptococcus agalactiae* isolates, from raw data to interpretation.

Step 1 — check your input data

Your basecalled, demultiplexed reads, one directory per sample:

```
ls ~/run_2026_03/fastq_pass/
# barcode01 barcode02 barcode03
ls ~/run_2026_03/fastq_pass/barcode01/ | head -3
# FBD86797_pass_barcode01_0.fastq.gz
```

Step 2 — find the species name

```
nextflow run nakast.nf --list-species | grep -i agalactiae
# sagalactiae    1    7  yes  Streptococcus agalactiae
```

Step 3 — write the samplesheet

Three tab-separated columns:

```
printf 'sample_id\tfastq_dir\tspecies\n' > samplesheet.tsv
printf 'SGB11\t%s/barcode01\tsagalactiae\n' ~/run_2026_03/fastq_pass >>
samplesheet.tsv
printf 'SGB12\t%s/barcode02\tsagalactiae\n' ~/run_2026_03/fastq_pass >>
samplesheet.tsv
printf 'SGB13\t%s/barcode03\tsagalactiae\n' ~/run_2026_03/fastq_pass >>
samplesheet.tsv
```

Use absolute paths. Relative paths resolve against the directory you launch from.

Step 4 — run

```
nextflow run /path/to/nakast/nakast.nf \
  --samplesheet samplesheet.tsv \
  --outdir results_run2026_03
```

Expect roughly 5–10 minutes for three samples on a laptop, most of it in KMA. The first run adds a few minutes to build the Conda environments.

Step 5 — read the report

Sample	ST	CC	adhP	pheS	atr	glnA	sdhA	glcK	tkr	Profile
SGB11	10	cc12	9	1	4	1	3	3	2	OK
SGB12	24	cc452	5	4	4	3	2	3	3	OK
SGB13	2	cc1	1	1	3	1	1	2	2	OK

Three isolates typed: ST10, ST24 and ST2. Section 3.5 explains what to do when a row does not look like these.

Step 6 — check the QC

Open `results_run2026_03/qc/multiqc_report.html` to confirm read counts and quality distributions are as expected for the run.

3.2 Input specification

A tab-separated file with a header row and three columns.

Column	Required	Description
sample_id	yes	Unique identifier. Becomes the output file prefix
fastq_dir	yes	Directory containing .fastq.gz or .fastq files
species	yes	Short name from the catalogue, for example <i>sagalactiae</i>

Additional columns are ignored, so samplesheets written for earlier versions keep working.

Validation runs before any heavy work and checks that the three columns exist, that `sample_id` is non-empty and unique, that the referenced directories exist on disk, and that `species` appears in `assets/pubmlst_species.tsv`. A misspelled species fails in seconds with a suggestion rather than after a partial download.

A mixed-species batch is written exactly the same way:

sample_id	fastq_dir	species
SGB11	/data/run1/barcode01	<i>sagalactiae</i>
SAL07	/data/run1/barcode02	<i>salmonella</i>

3.3 Parameter reference

Input and output

Parameter	Default	Description
--samplesheet	<code>samplesheet.tsv</code>	Input table
--outdir	<code>results</code>	Base output directory
--qc_outdir	<code><outdir>/qc</code>	QC outputs
--consensus_outdir	<code><outdir>/consensus_sequences</code>	Consensus FASTA
--report_outdir	<code><outdir>/reports</code>	.res files and profile tables
--mlst_report	<code>mlst_profiles</code>	Base name of the final report

Read filtering

Parameter	Default	Description
--nano_base_quality	<code>20</code>	Minimum Phred quality, passed to chopper
--min_length	<code>300</code>	Minimum read length (filtlong)
--keep_percent	<code>80</code>	Keep this percentage of the best reads by bases (filtlong)

Allele calling

Parameter	Default	Description
--min_depth	<code>30</code>	Minimum depth, applied in KMA and again in the report script
--min_identity	<code>90</code>	Minimum identity to the reference allele (%)
--allele_coverage	<code>90</code>	Minimum allele (template) coverage (%)
--kma_k	<code>31</code>	K-mer size for the KMA index
--generate_consensus	<code>true</code>	Publish per-locus consensus sequences

Run modes

Parameter	Default	Description
--qc_only	<code>false</code>	Run QC only; no download, no typing
--list_species	<code>false</code>	Print the supported-species catalogue and exit
--pubmlst_anonymous	<code>false</code>	Query PubMLST without a key; data up to 31 Dec 2024 only (section 2.4)

Parameter	Default	Description
--help	false	Print usage and exit

Nextflow options worth knowing

These belong to Nextflow itself and take a single dash.

Option	Purpose
-resume	Reuse completed work from the previous run
-profile host	Use the tools on your PATH instead of Conda
-with-report	Extra HTML execution report

Resources

Assigned by label in nextflow.config. Edit that file to fit your machine.

Label	CPUs	Memory	Time	Processes
process_low	2	4 GB	2 h	validation, database, species listing
process_medium	6	10 GB	8 h	concatenation, filtering, QC, report
process_high	12	16 GB	24 h	KMA alignment

3.4 Output reference

results/	
├ mlst_profiles_<species>.txt	← the answer
├ mlst_profiles_<species>.xlsx	
├ qc/	
│ └ qc_<sample>/	
│ └ multiqc_report.html	← run quality
├ consensus_sequences/	
│ └ <sample>.fsa	
├ reports/	
│ └ <sample>.res	← evidence per allele
│ └ <species>_profiles.tsv	
└ pipeline_info/	

Which file answers which question

Question	File
What ST is my isolate?	mlst_profiles_<species>.txt
Why was this allele called?	reports/<sample>.res
Did the run sequence well?	qc/multiqc_report.html
What sequence do I submit to PubMLST or confirm by Sanger?	consensus_sequences/<sample>.fsa
Which PubMLST version did this run use?	reports/<species>_profiles.tsv
How long did it take, what failed?	pipeline_info/

Final report columns

Column	Content
Sample	Sample identifier
ST	Sequence Type, or - if the profile matches nothing
CC	Clonal Complex, or - if unassigned or not defined by the scheme
one column per locus	Allele call with its quality annotation
Profile	OK, or INCOMPLETE (n/N) when fewer than N loci were recovered

Intermediate files

- `reports/<sample>.res` — raw KMA output. The authoritative record of identity, coverage, depth and score for every template considered.
- `reports/<species>_profiles.tsv` — the exact PubMLST profile table used. Keep it: PubMLST changes over time, and this file makes the run reproducible after the fact.
- `consensus_sequences/<sample>.fsa` — KMA consensus per locus.
- `pipeline_info/` — execution report, timeline and trace.

3.5 Interpreting your results

Work through this in order when a row is not a clean OK with an ST.

The allele annotations

Notation	Meaning
9	Exact match to allele 9
~9	Closest to allele 9, but the sequence differs
9?	Matches allele 9, but the read is longer than the reference
INS	The read covers less than the full allele
-	No call: below identity, coverage or depth thresholds

Only bare integers can match a PubMLST profile. If any locus carries ~, ?, INS or -, the profile cannot match and ST will be -. The ST lookup did not fail; the profile was simply incomplete.

Decision guide

ST = - but every locus has a bare integer. The profile is complete but that combination is not in PubMLST. This is a genuinely new ST. Consider submitting it.

ST = - and one locus shows ~N. The most common case. That locus differs from every known allele. Either it is a novel allele, or the consensus carries a sequencing error. To tell them apart, check the depth of that locus in the `.res` file. A consistent difference at high depth points to a real novel allele; confirm by Sanger using the consensus in `consensus_sequences/`. A difference at low depth is more likely an artefact.

ST = - and one locus shows -. That locus was not recovered. Check its depth in the .res file. Depth below --min_depth means the amplicon underperformed, which is a wet-lab problem, not an analysis one. Re-amplify or sequence deeper.

Profile = INCOMPLETE (6/7). Six of seven loci were recovered. Same diagnosis as above: find the missing locus and check its depth.

ST assigned but CC = -. Usually correct. Many STs have no clonal complex, and 33 of the 135 schemes define none at all. Check the CC column in the catalogue for your species.

Every sample has ST = -. Suspect the species value before suspecting the data. Typing against the wrong scheme produces empty results without any error.

Reading the evidence file

```
column -t results/reports/SGB20.res | head
```

The columns that matter are #Template (locus and allele), Template_Identity, Template_Coverage, Query_Coverage and Depth. An allele called ~9 will show Template_Identity below 100. A locus reported as - may be absent from the file entirely, which means nothing passed the thresholds.

3.6 Assessing data quality

NAKAST does not enforce a quality gate beyond --min_depth. Use these reference points from validation on the in-house *S. agalactiae* amplicon panel.

Metric	Typical	Concerning
Depth per locus	several hundred to a few thousand	below 100
Loci recovered	7 of 7	6 or fewer
Annotated alleles per sample	0	2 or more
Median read quality	around Q18	below Q12

Depth varies a lot between loci in an amplicon panel: a ten-fold spread between the best and worst amplicon is normal. What matters is that the weakest locus clears --min_depth.

Very high depth does not rescue a bad amplification. If one locus is consistently weak across every sample in a run, the cause is the primer or the reaction, not the analysis.

4. Technical module: administration and maintenance

This section is for whoever administers, adapts or maintains the pipeline. It describes how it works internally, what it assumes about the data, and where its limits lie.

4.1 Workflow architecture

Process	Tool	Purpose
VALIDATE_SAMPLESHEET	python, pandas	Pre-flight check of the samplesheet
DOWNLOAD_PREP_DB	python, kma	Detect scheme, download alleles and profiles, build index
concat_ONT_fastq	coreutils, gzip	Merge a barcode directory into one FASTQ
FILTER_ONT	filtlong, chopper	Length and quality filtering
NANOPLLOT	NanoPlot	Per-sample read QC
MULTIQC	MultiQC	Aggregate QC report
KMA_RUN	kma	Align reads against the allele database
GENERATE_REPORT	python, pandas, openpyxl	Allele calling, ST/CC assignment, reports
LIST_SPECIES	python	Print the supported-species catalogue

Failure behaviour differs by stage, on purpose:

Process	Strategy	Rationale
VALIDATE_SAMPLESHEET	terminate	A bad samplesheet should stop everything immediately
DOWNLOAD_PREP_DB	retry ×3, then finish	Survives transient network failures; lets QC finish so -resume can continue
KMA_RUN, GENERATE_REPORT	terminate	A silent failure here would produce a run with no results and exit 0
concat_ONT_fastq, FILTER_ONT, NANOPLLOT, MULTIQC	ignore	One bad sample should not abort the batch

4.2 How allele calling works

Step 1 — depth filter

Templates below --min_depth are discarded, both by KMA and again in the report script.

Step 2 — pick a winner per locus

Several alleles of the same locus usually attract reads, since they differ by a few bases. NAKAST ranks them with a confidence score:

```
Score_Ratio = Score / Expected
Confidence = Score_Ratio × log1p(Depth) × perfect_bonus
```

perfect_bonus is 1.5 when identity, template coverage and query coverage are all 100%, and 1.0 otherwise. The highest-confidence template wins the locus.

The bonus is multiplicative rather than absolute on purpose: a perfect match still needs depth behind it, so a single well-aligned read cannot outrank a deeply covered allele.

Step 3 — annotate the call

Notation	Condition
9	Identity, template coverage and query coverage all 100%
~9	Query and template coverage 100%, identity below 100%
9?	Query coverage above 100%, identity and coverage above thresholds
INS	Query coverage below 100%
-	Identity or template coverage below thresholds, or no template passed

A note on ?: query coverage above 100% means the read is *longer* than the template. With an amplicon panel this reflects read-through rather than truncation, so the call is usually sound. Treat 9? as “probably allele 9, worth confirming” rather than as a failure.

4.3 ST and CC assignment

The allele calls are joined into a signature (9/1/4/1/3/3/2) and matched against the same signature built from the PubMLST profile table. The match is exact and string-based:

- Loci come from <species>_loci.txt, written from the scheme definition, then ordered as they appear in the profile table so the report preserves PubMLST’s canonical order.
- The profile table is read as text so a missing value cannot silently turn a locus column into floating-point numbers and produce 1.0/1/2/..., which would fail every match.
- The CC column is detected tolerantly (clonal_complex, clonal complex, cc, clonalcomplex). When absent, CC = -.
- No match returns ST = -. **NAKAST never reports a nearest or approximate ST.**

4.4 Species support

assets/pubmlst_species.tsv and docs/supported_species.md catalogue the **135 PubMLST schemes** that define ST profiles. Regenerate them with:

```
python3 bin/list_pubmlst_species.py --write-catalogue .
```

Two assumptions that hold for *S. agalactiae* but not in general, which NAKAST therefore resolves at run time:

- **The MLST scheme is not always scheme 1.** Seven schemes use a different id; *Salmonella* uses 2. NAKAST selects the scheme whose description starts with MLST.

- **Schemes do not always have seven loci.** They range from 2 to 10, and 48 of 135 differ from seven. The locus list comes from the scheme definition, never from a fixed list.

Locus names are also less regular than they look: MLST_adk, 16S_rRNA and int_hyp all contain underscores, so allele identifiers are split from the right.

4.5 Assumptions

These matter for interpreting results.

1. **Amplicon sequencing, not whole genome.** Reads are assumed to come from targeted amplification of the MLST loci. Depth is expected in the hundreds or thousands and the default thresholds reflect that. Whole-genome data will give far lower per-locus depth and many - calls.
2. **The species is known and declared.** NAKAST does not identify the organism; it types the sample against the scheme you specify. A wrong species produces an empty result, not an error.
3. **PubMLST is reachable and authoritative.** The database is downloaded fresh on every run, so results depend on the state of PubMLST at run time. That is why the profile table is published with the outputs. The complete scheme is only obtained with a valid access key (section 2.4).
4. **The scheme defines ST profiles.** Schemes without a primary key, such as cgMLST, are excluded from the catalogue and rejected at download time.
5. **Sample identifiers are unique** across the samplesheet. They name the output files.
6. **Reads are basecalled and demultiplexed.** NAKAST starts from per-barcode FASTQ. It does not basecall, demultiplex, or trim adapters and barcodes.
7. **One organism per sample.** Mixed or contaminated cultures produce competing alleles at some loci; the confidence score will pick one, and a ~ annotation is often the only visible hint.

4.6 Limitations

1. **Oxford Nanopore only.** There is no short-read path. Filtering and alignment parameters are tuned for the ONT error profile, and the samplesheet takes a directory of FASTQ files rather than read pairs.
2. **No novel allele detection or submission.** A sequence absent from PubMLST is reported as ~N against the closest known allele. NAKAST will not flag it as new, propose a number, or prepare a submission. Confirmation, by Sanger for instance, is a manual step; the consensus in consensus_sequences/ is the starting point.
3. **Network required.** There is no offline mode and no local cache between runs. Every run re-downloads the scheme.
4. **Incomplete data without a key.** With --pubmlst_anonymous only data deposited up to 31 December 2024 is used. For *S. agalactiae*, in September 2026, that left out 301 of 2673 STs (11%).
5. **Terms of use for recent data.** PubMLST data deposited since 2025, obtained with the key, may only be used for non-commercial academic research or public health sur-

veillance and may not be redistributed. Commercial use requires a licence from the University of Oxford. Data deposited before 2025 carries no such restriction.

6. **Biologically validated on one species.** The architecture is species-agnostic and was exercised against the *Salmonella*, *Brucella*, *Vibrio* and *Mycoplasma genitalium* schemes, but validation against known isolates was done only for *S. agalactiae* with the in-house amplicon panel. Alignment parameters are tuned for that panel.
7. **Clonal complexes depend on the scheme.** 33 of 135 schemes define none, and even where defined many STs have no CC.
8. **No contamination or mixed-sample detection.** There is no explicit check for multiple alleles at a locus.
9. **Depth thresholds are global.** `--min_depth` applies to every locus equally. A panel where one amplicon systematically underperforms may need the threshold lowered for the whole run.
10. **One report per species.** Batches spanning several species produce several files rather than one merged table, because the locus columns differ.

4.7 Reproducibility

Every process declares its own Conda environment with exact versions:

Process	Environment
VALIDATE_SAMPLESHEET	python=3.13.11 pandas=2.3.3
DOWNLOAD_PREP_DB, KMA_RUN	python=3.13.11 kma=1.6.8
concat_ONT_fastq	python=3.13.11 coreutils=9.5 gzip=1.12
FILTER_ONT	python=3.13.11 filtlong=0.3.1 chopper=0.12.0 gzip=1.12
NANOPLLOT	python=3.13.11 nanoplot=1.46.2
MULTIQC	python=3.13.11 multiqc=1.33
GENERATE_REPORT	python=3.13.11 pandas=2.3.3 numpy=2.4.0 openpyxl=3.1.5
LIST_SPECIES	python=3.13.11

Environments are cached in `$HOME/.nextflow/conda` and built with mamba. Identical specifications share one environment.

The one input that is *not* pinned is PubMLST itself, which changes as curators add alleles and STs. To make a past run reproducible, keep `reports/<species>_profiles.tsv` and the `pipeline_info/` trace alongside the report.

4.8 Design notes

Two findings from tuning against real *S. agalactiae* data are recorded here because they are counter-intuitive and easy to reintroduce.

KMA's `-mrc` is not allele coverage. `-mrc` is *minimum query coverage*: the fraction of the **read** that must align. An earlier version passed `--allele_coverage` to it. With reads averaging around 950 bp against 500 bp amplicons, most reads cannot reach 90% query cover-

age. Loci with abundant reads survived anyway, but weakly amplified loci lost every read and vanished despite being perfect matches at 100% identity and coverage. The flag was removed; allele coverage is enforced downstream on `Template_Coverage`, where it belongs.

filtlong's quality thresholds are not on the Phred scale. `--min_mean_q` and `--min_window_q` take a 0–100 accuracy percentage, so passing 20 filters nothing at all (Q20 corresponds to 99). Setting them to a value that does filter proved harmful: at 95 the weakly amplified loci lost all support, because low-abundance loci also tend to carry the lowest-quality reads. The flags were removed and quality is controlled solely by chopper, on the Phred scale, through `--nano_base_quality`.

A related negative result: escalating quality tiers (Q30 → Q25 → Q20) is not viable for typical ONT amplicon data. With a median per-read quality near Q18, thresholds above roughly Q22 retain a fraction of a percent of bases, and the surviving fragments are shorter than a single amplicon, so they cannot cover an allele.

5. Support, citation and licence

5.1 Reporting problems

Open an issue at <https://github.com/javiertognarelli/nakast/issues>. Include the Nextflow version (`nextflow -version`), the command you ran, and the relevant part of `.nextflow.log`.

5.2 Authorship

Javier Alejandro Tognarelli Santiago — Genómica UV, Escuela de Medicina, Universidad de Valparaíso, Chile

Daniel Fernando Escobar Araya — Unidad de Investigación e Innovación, Instituto de Salud Pública, Chile

Fernando Andrés Amaya Inzunza — Unidad de Investigación e Innovación, Instituto de Salud Pública, Chile

5.3 How to cite NAKAST

Copy the form that matches the style of your document.

Vancouver style

Tognarelli Santiago JA, Escobar Araya DF, Amaya Inzunza FA. NAKAST: MLST typing from Oxford Nanopore amplicon sequencing [software]. Version 1.0. Valparaíso: Universidad de Valparaíso; 2026. Available from: <https://github.com/javiertognarelli/nakast>

APA style (7th edition)

Tognarelli Santiago, J. A., Escobar Araya, D. F., & Amaya Inzunza, F. A. (2026). *NAKAST: MLST typing from Oxford Nanopore amplicon sequencing* (Version 1.0) [Computer software]. Universidad de Valparaíso. <https://github.com/javiertognarelli/nakast>

BibTeX, for reference managers and LaTeX:

```
@software{nakast2026,  
  author    = {Tognarelli Santiago, Javier Alejandro and  
              Escobar Araya, Daniel Fernando and  
              Amaya Inzunza, Fernando Andrés},  
  title     = {{NAKAST}: {MLST} typing from {Oxford Nanopore} amplicon sequencing},  
  version   = {1.0},  
  year      = {2026},  
  publisher = {Universidad de Valparaíso},  
  url       = {https://github.com/javiertognarelli/nakast}  
}
```

5.4 References for the components

A publication using NAKAST results should also cite the data source and the tools it relies on. Vancouver-style references, ready to copy:

1. Jolley KA, Bray JE, Maiden MCJ. Open-access bacterial population genomics: BIGSdb software, the PubMLST.org website and their applications. *Wellcome Open Res.* 2018;3:124. doi:10.12688/wellcomeopenres.14826.1
2. Clausen PTL, Aarestrup FM, Lund O. Rapid and precise alignment of raw reads against redundant databases with KMA. *BMC Bioinformatics.* 2018;19(1):307. doi:10.1186/s12859-018-2336-6
3. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol.* 2017;35(4):316-319. doi:10.1038/nbt.3820
4. De Coster W, Rademakers R. NanoPack2: population-scale evaluation of long-read sequencing data. *Bioinformatics.* 2023;39(5):btad311. doi:10.1093/bioinformatics/btad311
5. Ewels P, Magnusson M, Lundin S, Käller M. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics.* 2016;32(19):3047-3048. doi:10.1093/bioinformatics/btw354
6. Wick R. Filtlong [software]. Available from: <https://github.com/rrwick/Filtlong>

Reference 1 is PubMLST, 2 is KMA, 3 is Nextflow, 4 covers NanoPlot and chopper, and 5 is MultiQC. Filtlong has no associated publication and is cited by its repository.

5.5 Third-party components and data source

The complete list of third-party components, with their versions and licences, is in [THIRD_PARTY_LICENSES.md](#).

Allele and profile definitions come from PubMLST (<https://pubmlst.org>), hosted at the University of Oxford. Its terms of use require the following acknowledgement, verbatim, in any publication based on its data, in addition to citing reference 1 of section 5.4:

This publication made use of the PubMLST website (<https://pubmlst.org/>) sited at the University of Oxford. The development of that website was funded by the Wellcome Trust.

6. Frequently asked questions

Do I need to install KMA, NanoPlot or other tools separately?

No. Each process declares its own Conda environment and Nextflow builds it automatically on the first run. You only need Nextflow and Conda (section 2.1).

Do I need a PubMLST account?

Yes, to obtain the complete scheme. The account is free; it is where you create the access key NAKAST uses (section 2.4). Without one you can run with `--pubmlst_anonymous`, but only with data deposited up to 31 December 2024.

Can I analyse several species in one run?

Yes. Declare each sample's species in the `species` column. NAKAST downloads one database per species and produces one report for each.

How do I know which name to use in the species column?

Run `nextflow run nakast.nf --list_species` and use the exact short name in the first column. The full catalogue is also in `docs/supported_species.md`.

Why does my sample show ST = -?

Almost always because one locus is annotated (~N, N?, INS or -), so the profile cannot match any in PubMLST. [Section 3.5](#) explains what to do in each case.

What does it mean when an allele shows as ~9?

That the closest allele is 9 but the sequence does not match it exactly. It may be a novel allele or a sequencing error; if the difference is consistent at high depth, confirm it by Sanger.

Every sample shows ST = -. What should I check first?

The `species` value. Typing against the wrong scheme produces empty results without any error. Then check the allele columns.

Can I use Illumina data?

No. NAKAST processes Oxford Nanopore reads only and its parameters are tuned for that error profile.

Does it work offline?

No. Every run downloads the current scheme from PubMLST.

The PubMLST download failed. Do I lose my progress?

No. The pipeline retries three times, lets in-flight quality control finish, and stops. Once connectivity is back, re-run the same command with `-resume`; completed work is reused.

-resume reuses nothing. Why?

Something in the task inputs changed between runs. If you modified the code, note that interpolating a Path object such as `launchDir` directly into a process script gives it an unstable hash; it must be passed as a `val` input.

A sample is missing from the report.

Its filtering or QC step failed and was ignored so the rest of the batch could continue. Look for `Error is ignored` in the Nextflow log and check `pipeline_info/` for which process failed.

Validation rejects the species.

The name is not in the catalogue. Run `--list_species` and use the exact short name; the error message suggests near matches.

I get No PubMLST API key found.

No key is stored in the Nextflow secrets store. Store one as described in section 2.4, or add `--pubmlst_anonymous` to run with data up to 2024 only.

I get PubMLST rejected the API key (HTTP 401).

The stored key is invalid or was revoked. Create a new one in your PubMLST account and store it again with the command in section 2.4; the old value is replaced.

I get Unknown configuration profile: 'conda'.

Conda is enabled by default. Drop the `-profile conda` flag from the command.

I get command not found or exit status 127.

The process ran against the host PATH instead of its Conda environment. Confirm that `nextflow config .` reports `conda { enabled = true }`, and that no stray `nextflow.config` in your launch directory is overriding it.

I get Variable declarations cannot be mixed with config statements.

A `nextflow.config` in your launch directory uses syntax that Nextflow 26 rejects. Nextflow merges that file with the pipeline's own, so an unrelated config from another project breaks the run. Launch from a clean directory.

How do I reproduce an old run exactly?

Keep `reports/<species>_profiles.tsv` and the `pipeline_info/` trace alongside the report. The tools are pinned; the only thing that changes over time is PubMLST, and that file records the version used.